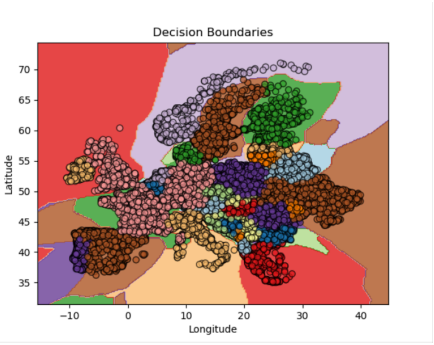
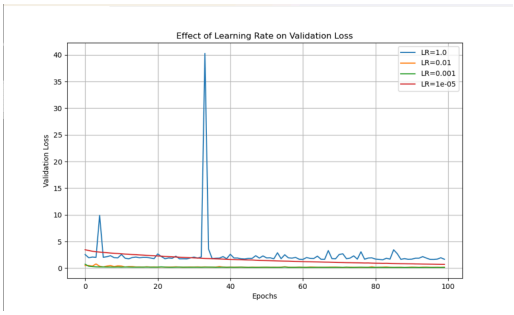
Report ex4

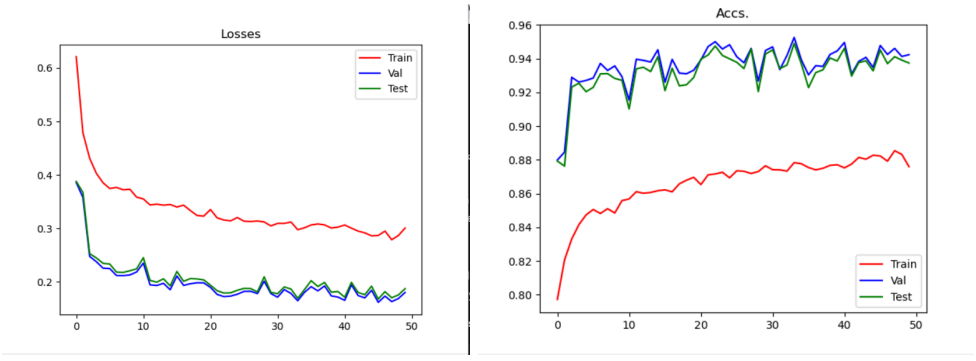
6.1.1 

The decision boundary plot confirms that the model is learning correctly. Instead of random guesses, we see clear colored regions that match the clusters of data points (cities). The boundaries separate different countries reasonably well, proving the code and model are working as intended.

6.1.2

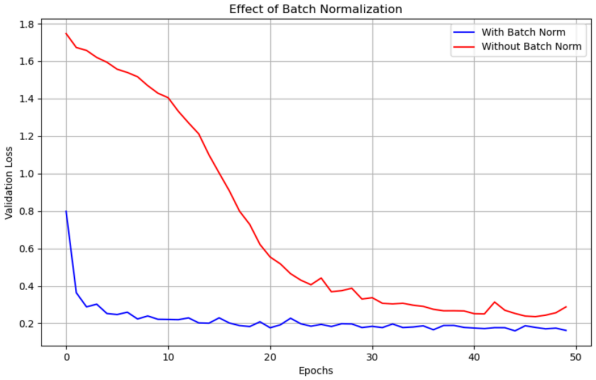
1) 

When you look at the graph, you will see four very different behaviors. The extremely high learning rate (1.0) causes the loss to jump around unpredictably or even increase because the model is "overshooting"—taking such huge steps that it constantly misses the target. On the other extreme, the tiny learning rate (0.00001) looks like a flat line; the model is taking microscopic steps, so while it is technically learning, it is far too slow to be useful. The moderate rates (0.01 and 0.001) represent the ideal balance, dropping steeply at the start and then smoothing out, showing that the model is learning efficiently and converging to a good solution.

2) 

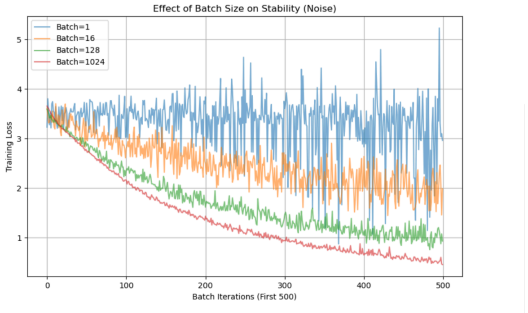
The graph demonstrates the balance required in training duration. Initially, all loss curves drop rapidly as the model learns the basic patterns of the map. However, as the epochs increase, you will notice the Training Loss (red) continues to get better and better, while the Validation Loss (blue) eventually flattens out or even starts to rise. This separation

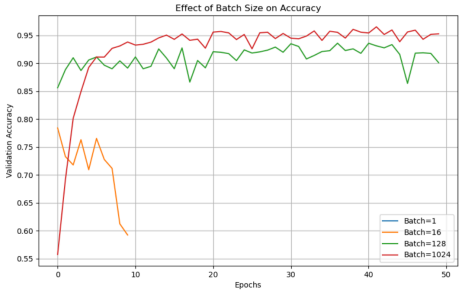
indicates overfitting: after too many epochs, the model stops learning the general rules and starts merely memorizing the specific training examples, which hurts its performance on new, unseen data. Conversely, if you use too few epochs, the model stops while the loss is still high, resulting in underfitting—it simply hasn't had enough time to find the solution.



3) Observation The model with Batch Normalization learns significantly faster than the regular model. The graph shows that the loss for the Batch Norm model drops sharply almost immediately and stabilizes at a low value. in contrast, the regular model without Batch Normalization improves much more slowly and exhibits a bumpier, less stable curve as it struggles to converge.

Explanation Batch Normalization improves training by solving a problem called Internal Covariate Shift. As the network learns, the weights in the early layers keep changing, which causes the distribution of inputs to the later layers to constantly shift around. Without Batch Norm, the later layers have to waste time re-adjusting to these constantly changing numbers. Batch Normalization fixes this by forcing the inputs of every layer to be stable, centered, and consistent. This stability prevents the values from getting too large or too small, allowing the model to learn smoothly and rapidly without getting stuck.





4)

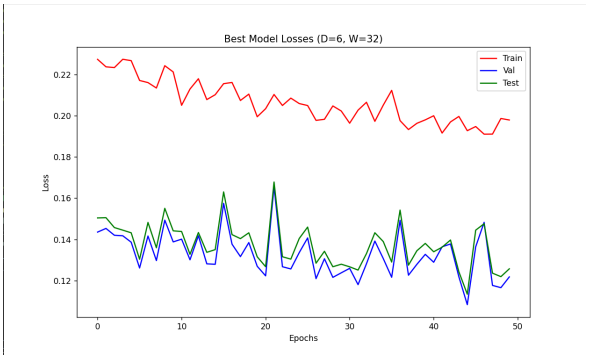
(i) Accuracy (Validation Graph)- The graph shows that smaller batch sizes, like 16 or 128, often achieve higher accuracy faster than the very large batch size of 1024. This happens because small batches introduce a bit of random noise into the training process. This noise acts like a

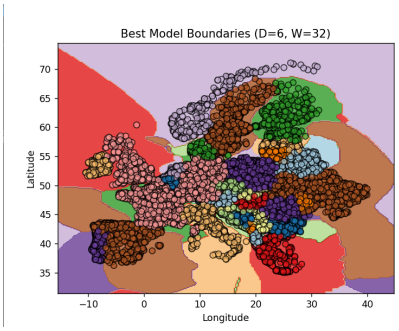
helpful "shake" that prevents the model from getting stuck in suboptimal solutions, allowing it to explore and find better final weights, whereas the large batch takes a safer, "averaged" path that learns more slowly.

(ii) Speed (Iterations per Epoch)- The speed difference is driven by how often the computer has to update the model. With a Batch Size of 1, the model updates over 33,000 times per epoch (once for every single city), which is incredibly slow. In contrast, with a Batch Size of 1024, the model groups the data into huge chunks and only needs about 32 updates to finish an entire epoch, making it significantly faster to run.

(iii) Stability (Loss vs. Batch Graph)- The stability graph illustrates a massive difference in "noise." The line for Batch Size 1 is thick and jagged because the model reacts violently to every single data point; if one city is an outlier, the error spikes immediately. The line for Batch Size 1024 is smooth and thin because it averages the error of over a thousand cities at once, causing individual outliers to cancel each other out and resulting in a very stable estimate of the loss.

6.2

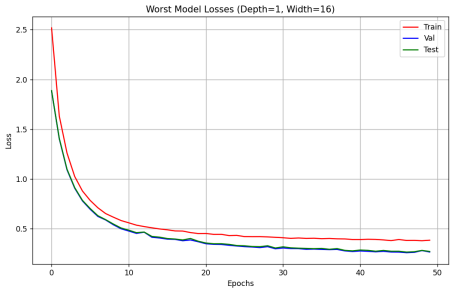
1) 

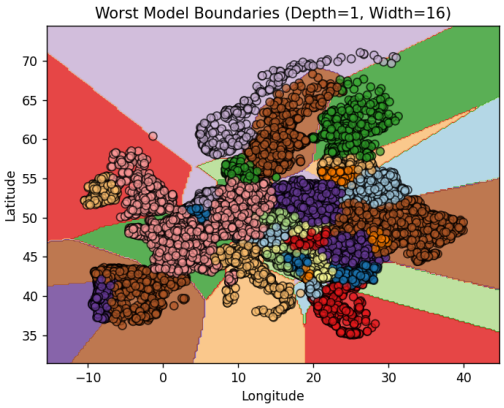


WINNER: Depth=6, Width=32 with Acc=0.9489

Yes, Explanation:

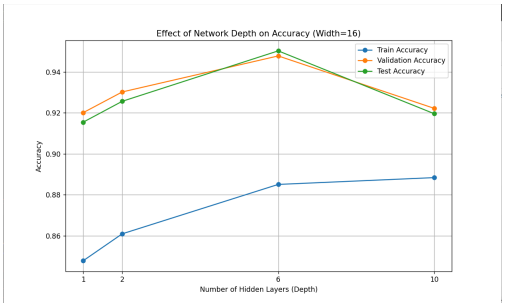
1. Smooth Boundaries: The decision boundary plot shows smooth, logical curves rather than jagged, messy shapes. This proves the model learned the general geography instead of just memorizing individual dots.
2. Consistent Loss: The Validation (Blue) and Test (Green) loss curves stay low and close to the Training (Red) curve. This confirms that the model performs just as well on new, unseen data as it does on the training data.

2) 

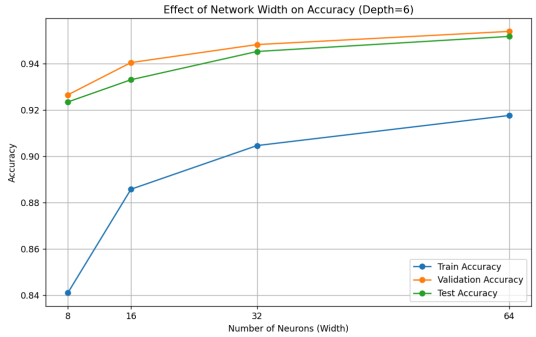



```
--- Training Worst Model (Depth=1, Width=16) --- Final Accuracy: 0.9201
```

This model is a clear case of under-fitting. You can see this in the loss graph, where the training and validation lines track each other almost perfectly and stop improving early. They hit a "performance wall" because the model is too simple to learn the data any better. The decision boundary plot confirms this lack of complexity: instead of drawing the curved, organic borders of real countries, the model is forced to cut the map with straight lines and sharp angles. It simply lacks the depth required to capture the true shapes of the dataset.

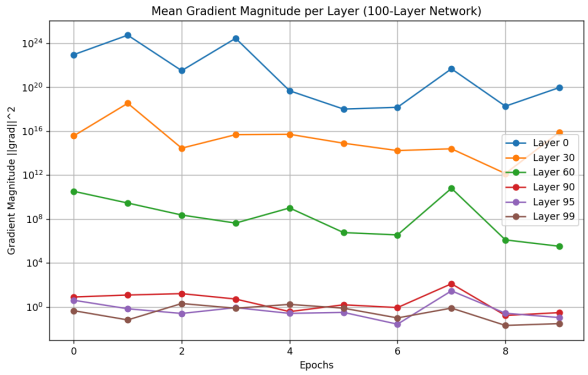
3) 

The graph shows that a depth of 6 layers works best. Increasing the depth from 1 to 6 improves accuracy because deeper networks have the power to learn the complex, curved borders of the countries. However, adding too many layers (like 10) actually makes the results worse. This is because very deep networks are much harder to train; the learning signal often gets lost before it reaches the first layers (vanishing gradients), making it difficult for the model to improve.



4) The graph clearly shows that widening the network significantly improves accuracy, especially at the beginning. We see a massive jump in performance when moving from 8 to 16 neurons. This happens because a width of 8 acts as a "bottleneck"; the layer is too narrow to pass all the necessary information through, forcing the model to lose important details.

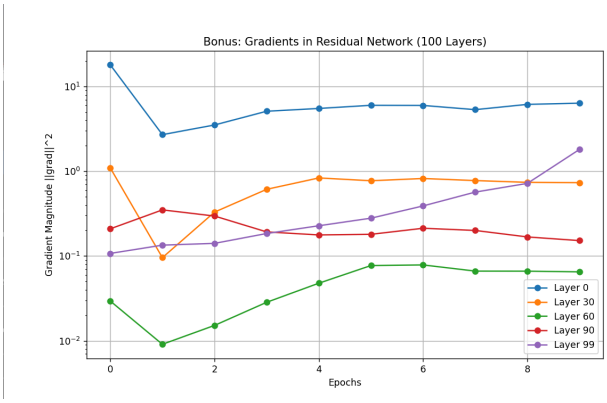
As we widen the layers to 32 and 64, the model gains the "capacity" to process different patterns simultaneously, which solves the bottleneck issue and boosts accuracy. However, the curve flattens out towards the end (between 32 and 64), indicating diminishing returns—once the network is wide enough to handle the data, adding even more neurons provides only minimal improvement.



5) Observation: The graph reveals a clear case of Exploding Gradients. We can see that the gradients in the first layer (Layer 0, Blue line) are astronomical—reaching values around 10^{24}—while the final layers (Layer 99) are relatively small.

Explanation: This occurs because the error signal gets multiplied by the network's weights repeatedly as it travels backward from the output to the input. In a network this deep (100 layers), even slightly large weights cause the signal to multiply into massive numbers by the time it reaches the start, making the training unstable.

Solution: We can modify the architecture by adding Residual Connections (Skip Connections). These connections act as "shortcuts" or highways that allow the gradient signal to flow directly to the earlier layers without passing through every single multiplicative layer, effectively preventing the signal from exploding or vanishing.

6) 

Yes, the modification solved the problem completely. In the previous experiment, we observed exploding gradients, where the signal in the early layers blew up to astronomical values (10^24).

By implementing Residual Connections (Skip Connections), we provided "shortcuts" or highways for the gradient to flow backward. This prevents the signal from being repeatedly multiplied into massive numbers as it passes through the layers. The new graph confirms this: the gradients for all layers (from 0 to 99) are now strictly contained within a healthy, stable range (around 10^-1 to 10^1), allowing the deep network to train effectively without instability.

7)1. Best and Worst Models per Baseline:

- XGBoost:
  - Test Accuracy: 73.50%
- Training from Scratch:
  - Best: LR 0.001 (55.50%) and LR 0.1 (54.00%)
  - Worst: LR 0.01 (48.00%)
- Linear Probing:
  - Best: LR 0.1 (74.00%) and LR 0.01 (72.00%)
  - Worst: LR 0.0001 (53.00%)
- Fine-tuning:
  - Best: LR 0.0001 (83.75%) and LR 0.001 (74.75%)
  - Worst: LR 0.01 (52.50%)

LR we used [1e-1, 1e-2, 1e-3, 1e-4, 1e-5].
2/Fine-tuning (The Winner)

Fine-tuning performed best with a low learning rate of 0.0001 (10^(-4)). This low rate allowed the model to gently adapt its pre-trained ImageNet knowledge to the new task without overwriting useful features. In contrast, higher learning rates (like 0.1) caused drastic updates that destroyed the pre-trained weights, leading to a collapse in accuracy.

Linear Probing

Linear Probing achieved its best results with a high learning rate of 0.1 (10^-1). Since the backbone is frozen and only the final layer is trained, the optimization process is highly stable. This stability allowed the model to take larger steps (higher learning rate) and converge quickly to a good solution.

Training from Scratch (The Loser)

Training from scratch yielded poor results across all learning rates. Deep networks like ResNet18 require many epochs to learn features from random initialization. Restricted to only a single epoch, the model simply did not have enough time to learn meaningful patterns from the data.

bonus:

```
re-training BEST model to capture predictions...
100%|██████████| 44/44 [06:53<00:00,  9.39s/it]
re-training WORST model to capture predictions...
100%|██████████| 44/44 [06:47<00:00,  9.26s/it]
Searching for images...
```



The visualization highlights the power of transfer learning. The Fine-Tuning model correctly classified these images because it started with a "pre-trained brain" capable of recognizing complex facial features immediately. Conversely, the model trained from scratch failed because it began with random weights and had only one epoch to learn; without sufficient time to understand visual patterns, it was effectively guessing on samples that the pre-trained model solved easily.